SQL-Dokumentation, Installationsanleitung und Testergebnisse

Miskovic, Dominik · Matrikelnummer: 14133980 · Kurs: DLBDSPBDM01_D · Finalisierungsphase

1. Dateien

Datei	Inhalt
01_schema.sql	DDL: 14 Tabellen, 14 Primär-, 17 Fremdschlüssel-, 7 UNIQUE- und 16 CHECK-Constraints, 31 Indizes, 8 Trigger, 6 Funktionen
02_dummy_data.sql	342 Datensätze, jede Tabelle mit mindestens 10 Einträgen (erzeugt mit generate_dummy_data.py)
03_testfaelle.sql	16 positive Abfragen und 21 Negativtests (jeweils BEGIN ... ROLLBACK)
04_metadaten.sql	Anzahl Tabellen, Einträge, Größen und Objektanzahlen
generate_dummy_data.py, run_tests.py	Hilfsskripte: Datengenerierung mit Regelprüfung, Testautomatisierung und Screenshot-Erzeugung

2. Installationsanleitung

Schritt 1: PostgreSQL installieren (ab Version 14; getestet mit 16)

```
macOS (Homebrew):  brew install postgresql@16 && brew services start postgresql@16
Debian/Ubuntu:     sudo apt install postgresql
Windows:           Installer von postgresql.org
```

Schritt 2: Datenbank anlegen

```
createdb buchtausch_app
```

Schritt 3: Schema einspielen (Tabellen, Constraints, Trigger, Indizes)

```
psql -d buchtausch_app -f 01_schema.sql
```

Schritt 4: Dummy-Daten einspielen

```
psql -d buchtausch_app -f 02_dummy_data.sql
```

Schritt 5: Testfälle ausführen (optional; Negativtests mit ROLLBACK)

```
psql -d buchtausch_app -f 03_testfaelle.sql
```

Schritt 6: Metadaten abrufen (optional)

```
psql -d buchtausch_app -f 04_metadaten.sql
```

Schritt 7: Automatisierte Tests mit Erwartungsprüfung (optional)

```
pip3 install pillow
python3 run_tests.py buchtausch_app
```

Die Dateien müssen in dieser Reihenfolge eingespielt werden, da 02_dummy_data.sql die Tabellen aus 01_schema.sql voraussetzt. 01_schema.sql löscht vorhandene Tabellen gleichen Namens (DROP TABLE IF EXISTS) und kann daher beliebig oft neu ausgeführt werden.

3. Geschäftsregeln und ihre Absicherung

Nr.	Regel	Mechanismus	Datenbankobjekt	Negativtest
1	E-Mail eindeutig und formal plausibel	UNIQUE + CHECK	benutzer_email_key, CHECK auf benutzer.email	neg_email
2	Höchstens eine Hauptadresse je Person	Partieller UNIQUE-Index	ux_adresse_hauptadresse	neg_hauptadresse
3	Gültige geografische Koordinaten	CHECK (5 Constraints)	chk_adresse_*, chk_abholort_*	neg_koordinaten
4	Erscheinungsjahr plausibel und nicht in der Zukunft	CHECK (feste Grenzen) + Trigger	buch.erscheinungsjahr, trg_buch_erscheinungsjahr	neg_erscheinungsjahr
5	Leihdauer eines Exemplars 1 bis 365 Tage	CHECK	exemplar_max_leihdauer_tage_check	neg_leihdauer_null
6	Besitzer eines Exemplars wechselt nicht	Trigger	trg_exemplar_besitzer_unveraenderlich	neg_besitzerwechsel
7	Zeitfenster: Ende nach Beginn	CHECK	chk_zeitfenster_reihenfolge	neg_zeitfenster_reihenfolge
8	Angebot nur mit Abholort und Zeitfenster des Besitzers	Trigger	trg_angebot_besitzer	neg_angebot_fremder_abholort
9	Gleiche Angebotskombination nur einmal	UNIQUE	UNIQUE (exemplar_id, abholort_id, zeitfenster_id)	neg_doppeltes_angebot
10	Keine Ausleihe des eigenen Exemplars	Trigger	trg_ausleihvorgang_regeln	neg_selbstausleihe
11	Geplante Dauer höchstens max_leihdauer_tage	Trigger	trg_ausleihvorgang_regeln	neg_leihdauer
12	Zeitfenster gehört dem Besitzer des Exemplars	Trigger	trg_ausleihvorgang_regeln	neg_fremdes_zeitfenster
13	Ausleihe nur über ein als Angebot eingerichtetes Zeitfenster	Trigger	trg_ausleihvorgang_regeln	neg_kein_angebot
14	Ein Exemplar nie mehrfach gleichzeitig offen ausgeliehen	Partieller UNIQUE-Index	ux_ausleihvorgang_offen_je_exemplar	neg_doppelte_ausleihe
15	Geplantes Rückgabedatum nach Ausleihdatum	CHECK	chk_ausleih_rueckgabe_reihenfolge	neg_rueckgabe_vor_ausleihe
16	Status 'zurueckgegeben' genau dann mit Rückgabedatum	CHECK	chk_ausleih_status_rueckgabe	neg_status_ohne_datum
17	Bewertung nur nach Abschluss, nicht vor Rückgabe	Trigger	trg_bewertung_nur_abgeschlossen	neg_bewertung_aktiv
18	Höchstens eine Bewertung je Ausleihvorgang	UNIQUE	bewertung_ausleihvorgang_id_key	neg_doppelte_bewertung
19	Sternebewertung 1 bis 5	CHECK	bewertung_sternebewertung_check	neg_sterne
20	Bewertete Ausleihe bleibt abgeschlossen	Trigger	trg_ausleihvorgang_status_nach_bewertung	neg_status_zurueck
21	Referenzielle Integrität	FOREIGN KEY (17 Stück)	alle Fremdschlüssel	neg_fremdschluessel

4. Umsetzung der Tutor-Rückmeldungen aus Phase 2

Rückmeldung	Umsetzung	Nachweis
CHECK auf erscheinungsjahr mit CURRENT_DATE	CHECK nur mit festen Grenzen (1450 bis 2100); die zeitabhängige Prüfung läuft in einem Trigger, der beim Schreiben ausgewertet wird.	neg_erscheinungsjahr

Rückmeldung	Umsetzung	Nachweis
Doppelte Ausleihen verhindern	Partieller UNIQUE-Index auf exemplar_id für Status 'aktiv' und 'ueberfaellig'.	neg_doppelte_ausleihe
Maximale Leihdauer prüfen	Trigger vergleicht geplante Dauer mit exemplar.max_leihdauer_tage.	neg_leihdauer
Zeitfenster und Angebote konsistent	Trigger prüfen: Abholort und Zeitfenster gehören dem Besitzer; Ausleihe nur über ein eingerichtetes Angebot; Besitzer-Spalten sind unveränderlich.	neg_fremdes_zeitfenster, neg_kein_angebot, neg_angebot_fremder_abholort
Bewerteten Status nicht zurücksetzbar	Trigger auf ausleihvorgang.status sperrt Rücksetzen, sobald eine Bewertung existiert.	neg_status_zurueck
Koordinaten und Status/Rückgabedatum	CHECK auf Breite/Länge (Adresse, Abholort) und CHECK: Status 'zurueckgegeben' genau dann, wenn ein Rückgabedatum gesetzt ist.	neg_koordinaten, neg_status_ohne_datum
Weitere Negativtests	21 Negativtests statt 2, je Geschäftsregel ein Test; zusätzlich Konsistenzprüfung.	alle neg_*
Mindestens 280 Datensätze	342 Datensätze (vorher 172), alle Tabellen mit mindestens 10 Einträgen.	Metadaten
Dokumentation der Absicherung	Tabelle 'Geschäftsregeln und ihre Absicherung' (BE, SQL-Dokumentation).	-

5. Ergebnisse: positive Testfälle

Jeder Screenshot zeigt die echte psql-Ausgabe des Testfalls gegen die befüllte Datenbank.

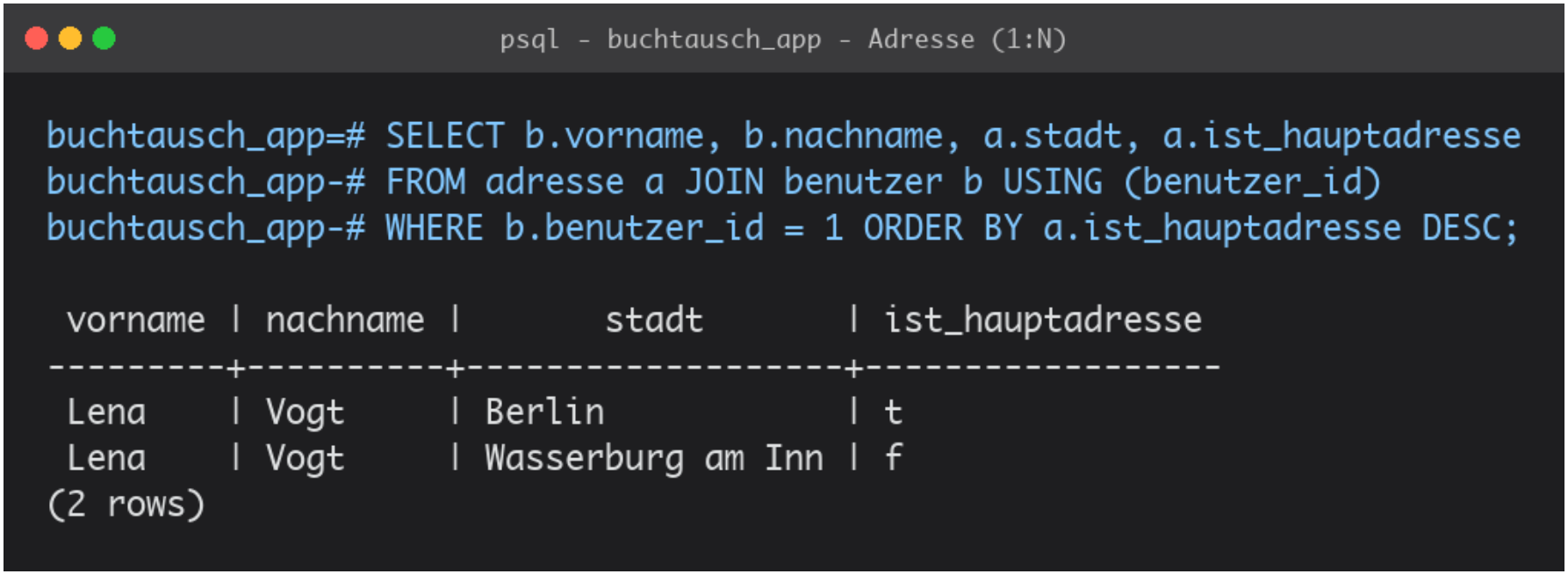
Benutzer

psql - buchtausch_app - Benutzer

```
buchtausch_app=# SELECT benutzer_id, vorname, nachname, email, registrierdatum
buchtausch_app-# FROM benutzer ORDER BY benutzer_id LIMIT 5;

 benutzer_id | vorname | nachname |           email            | registrierdatum
-------------+---------+----------+----------------------------+-----------------
           1 | Lena    | Vogt     | lena.vogt@example.com      | 2025-01-04
           2 | Karsten | Brandt   | karsten.brandt@example.com | 2025-01-07
           3 | Mira    | Falk     | mira.falk@example.com      | 2025-01-10
           4 | Jonas   | Hoffmann | jonas.hoffmann@example.com | 2025-01-13
           5 | Selin   | Aydin    | selin.aydin@example.com    | 2025-01-16
(5 rows)
```

Adresse (1:N)


psql - buchtausch_app - Adresse (1:N)
buchtausch_app=# SELECT b.vorname, b.nachname, a.stadt, a.ist_hauptadresse
buchtausch_app-# FROM adresse a JOIN benutzer b USING (benutzer_id)
buchtausch_app-# WHERE b.benutzer_id = 1 ORDER BY a.ist_hauptadresse DESC;
vorname | nachname | stadt | ist_hauptadresse
Lena | Vogt | Berlin | t
Lena | Vogt | Wasserburg am Inn | f
(2 rows)

Verlag


psql - buchtausch_app - Verlag
buchtausch_app=# SELECT verlag_id, name, land FROM verlag ORDER BY name LIMIT 5;
 verlag_id | name | land
-----------+--------------------+-------------
        11 | Aufbau Verlag | Deutschland
         9 | btb Verlag | Deutschland
        12 | Carl Hanser Verlag | Deutschland
         5 | Diogenes Verlag | Schweiz
         3 | dtv | Deutschland
(5 rows)

Genre


psql - buchtausch_app - Genre
buchtausch_app=# SELECT genre_id, bezeichnung FROM genre ORDER BY genre_id LIMIT 8;
 genre_id | bezeichnung
----------+-------------
        1 | Roman
        2 | Krimi
        3 | Fantasy
        4 | Sachbuch
        5 | Biografie
        6 | Kinderbuch
        7 | Lyrik
        8 | Thriller
(8 rows)

Sprache

psql - buchtausch_app - Sprache

```
buchtausch_app=# SELECT sprache_id, bezeichnung, iso_code FROM sprache ORDER BY bezeichnung LIMIT 5;

 sprache_id | bezeichnung | iso_code
------------+-------------+----------
          1 | Deutsch     | de
          2 | Englisch    | en
          3 | Französisch | fr
         12 | Griechisch  | el
          5 | Italienisch | it
(5 rows)
```

Buch

psql - buchtausch_app - Buch

```
buchtausch_app=# SELECT bu.titel, bu.erscheinungsjahr, g.bezeichnung AS genre, v.name AS verlag, s.bezeichnung AS sprache
buchtausch_app-# FROM buch bu JOIN genre g USING (genre_id) JOIN verlag v USING (verlag_id)
buchtausch_app-# JOIN sprache s USING (sprache_id) ORDER BY bu.titel LIMIT 5;

         titel         | erscheinungsjahr |   genre    |     verlag      | sprache
-----------------------+------------------+------------+-----------------+----------
 Backen mit Brot       |             2022 | Kochbuch   | Piper Verlag    | Deutsch
 Das Echo der Berge    |             2024 | Thriller   | btb Verlag      | Deutsch
 Der Gartenbriefkasten |             2013 | Kinderbuch | dtv             | Deutsch
 Der Glasbläser        |             2019 | Roman      | Rowohlt Verlag  | Deutsch
 Der letzte Leuchtturm |             2014 | Krimi      | Diogenes Verlag | Englisch
(5 rows)
```

Autor (N:M)

psql - buchtausch_app - Autor (N:M)

```
buchtausch_app=# SELECT bu.titel, string_agg(a.vorname || ' ' || a.nachname, ', ') AS autor_innen
buchtausch_app-# FROM buch bu JOIN buch_autor ba USING (buch_id) JOIN autor a USING (autor_id)
buchtausch_app-# GROUP BY bu.buch_id, bu.titel HAVING count(*) > 1 ORDER BY bu.titel;

       titel        |          autor_innen
--------------------+--------------------------------
 Das Echo der Berge | Lea Winter, Carla Reimann
 Der Glasbläser     | Samuel Kern, Hannes Rott
 Die Drachenkönigin | Carla Reimann, Sofie Lindqvist
 Die Kartografin    | Konrad Salm, Rebecca Thorn
 Die letzte Fähre   | Jens Dornbusch, Viktor Ruppert
 Salz und Meer      | Nadine Fuchs, Lea Winter
(6 rows)
```

Buch_Autor

psql - buchtausch_app - Buch_Autor

```
buchtausch_app=# SELECT buch_id, autor_id FROM buch_autor ORDER BY buch_id LIMIT 6;

 buch_id | autor_id
---------+----------
       1 |        1
       2 |        2
       3 |        3
       3 |        5
       4 |        4
       5 |        5
(6 rows)
```

Exemplar

psql - buchtausch_app - Exemplar

```
buchtausch_app=# SELECT e.exemplar_id, bu.titel, be.vorname || ' ' || be.nachname AS besitzer_in,
buchtausch_app-#        e.zustand, e.max_leihdauer_tage
buchtausch_app-# FROM exemplar e JOIN buch bu USING (buch_id)
buchtausch_app-# JOIN benutzer be ON be.benutzer_id = e.besitzer_id ORDER BY e.exemplar_id LIMIT 5;

 exemplar_id |         titel         |  besitzer_in   |  zustand  | max_leihdauer_tage
-------------+-----------------------+----------------+-----------+--------------------
           1 | Der Winterpfad        | Lena Vogt      | sehr gut  |                 14
           2 | Mord im Nebel         | Karsten Brandt | gut       |                 21
           3 | Die Drachenkönigin    | Mira Falk      | gebraucht |                 28
           4 | Wege durch die Stille | Jonas Hoffmann | neu       |                 30
           5 | Der letzte Leuchtturm | Selin Aydin    | sehr gut  |                 10
(5 rows)
```

Abholort

psql - buchtausch_app - Abholort

```
buchtausch_app=# SELECT abholort_id, bezeichnung, stadt, breitengrad, laengengrad
buchtausch_app-# FROM abholort ORDER BY abholort_id LIMIT 5;

 abholort_id |         bezeichnung          |       stadt       | breitengrad | laengengrad
-------------+------------------------------+-------------------+-------------+-------------
           1 | Treffpunkt Berlin            | Berlin            |   52.524000 |   13.409000
           2 | Treffpunkt Hamburg           | Hamburg           |   53.555000 |    9.998000
           3 | Treffpunkt München           | München           |   48.141000 |   11.580000
           4 | Treffpunkt Köln              | Köln              |   50.942000 |    6.964000
           5 | Treffpunkt Frankfurt am Main | Frankfurt am Main |   50.114000 |    8.686000
(5 rows)
```

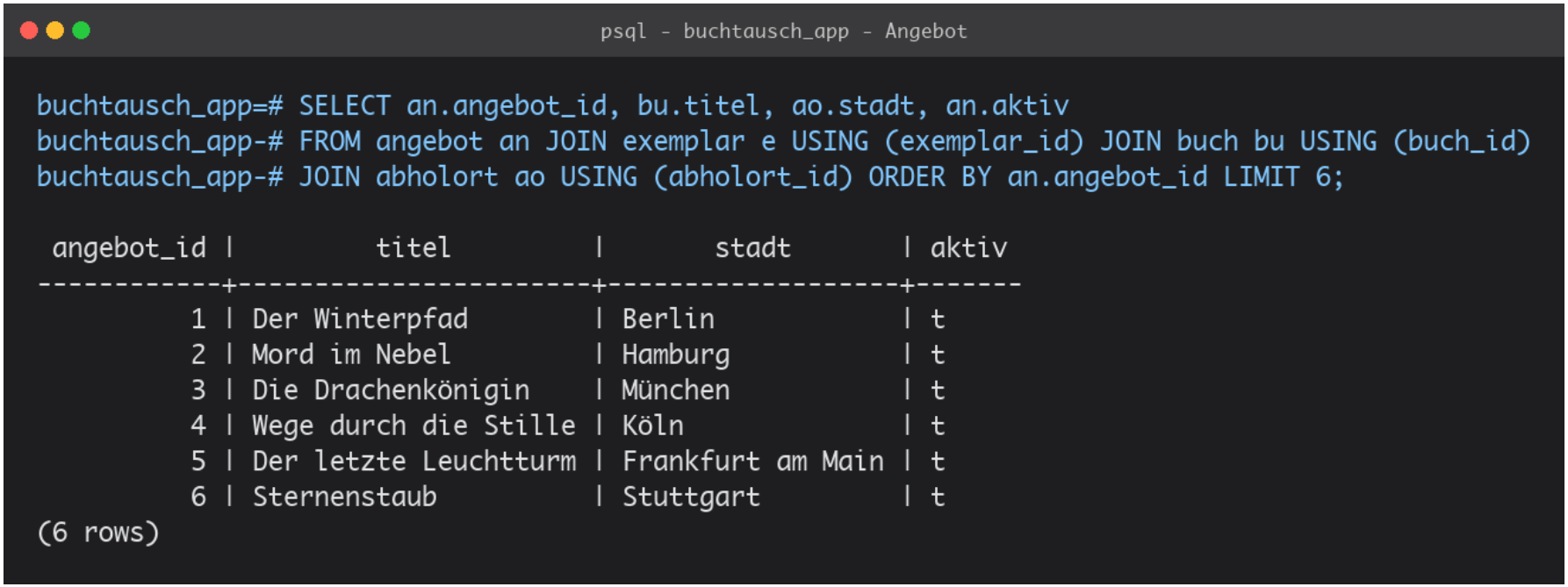
Verfuegbarkeitszeitfenster

psql - buchtausch_app - Verfuegbarkeitszeitfenster

```
buchtausch_app=# SELECT zeitfenster_id, benutzer_id, wochentag, startzeit, endzeit
buchtausch_app-# FROM verfuegbarkeitszeitfenster ORDER BY zeitfenster_id LIMIT 5;

 zeitfenster_id | benutzer_id | wochentag  | startzeit | endzeit
----------------+-------------+------------+-----------+----------
              1 |           1 | Montag     | 09:00:00  | 11:00:00
              2 |           2 | Dienstag   | 17:00:00  | 19:00:00
              3 |           3 | Mittwoch   | 09:00:00  | 11:00:00
              4 |           4 | Donnerstag | 17:00:00  | 19:00:00
              5 |           5 | Freitag    | 09:00:00  | 11:00:00
(5 rows)
```

Angebot


psql - buchtausch_app - Angebot
buchtausch_app=# SELECT an.angebot_id, bu.titel, ao.stadt, an.aktiv
buchtausch_app-# FROM angebot an JOIN exemplar e USING (exemplar_id) JOIN buch bu USING (buch_id)
buchtausch_app-# JOIN abholort ao USING (abholort_id) ORDER BY an.angebot_id LIMIT 6;
angebot_id | titel | stadt | aktiv
1 | Der Winterpfad | Berlin | t
2 | Mord im Nebel | Hamburg | t
3 | Die Drachenkönigin | München | t
4 | Wege durch die Stille | Köln | t
5 | Der letzte Leuchtturm | Frankfurt am Main | t
6 | Sternenstaub | Stuttgart | t
(6 rows)

Ausleihvorgang



Bewertung

psql - buchtausch_app - Bewertung

```
buchtausch_app=# SELECT bu.titel, ROUND(AVG(bw.sternebewertung), 1) AS durchschnitt, count(*) AS anzahl
buchtausch_app-# FROM bewertung bw JOIN ausleihvorgang av USING (ausleihvorgang_id)
buchtausch_app-# JOIN exemplar e USING (exemplar_id) JOIN buch bu USING (buch_id)
buchtausch_app-# GROUP BY bu.buch_id, bu.titel ORDER BY durchschnitt DESC, bu.titel LIMIT 6;

         titel          | durchschnitt | anzahl
------------------------+--------------+--------
 Der Gartenbriefkasten |          5.0 |      1
 Der Winterpfad        |          5.0 |      1
 Die Drachenkönigin    |          5.0 |      1
 Die Uhrmacherin       |          5.0 |      1
 Heimat im Sturm       |          5.0 |      1
 Mondfinsternis        |          5.0 |      1
(6 rows)
```

Raeumliche Suche (Berlin)

psql - buchtausch_app - Raeumliche Suche (Berlin)

```
buchtausch_app=# SELECT DISTINCT bu.titel, ao.stadt,
buchtausch_app-#        ROUND((6371 * acos(cos(radians(52.520)) * cos(radians(ao.breitengrad)) *
buchtausch_app-#          cos(radians(ao.laengengrad) - radians(13.405)) +
buchtausch_app-#          sin(radians(52.520)) * sin(radians(ao.breitengrad))))::numeric, 1) AS entfernung_km
buchtausch_app-# FROM angebot a JOIN exemplar e USING (exemplar_id) JOIN buch bu USING (buch_id)
buchtausch_app-# JOIN abholort ao USING (abholort_id) WHERE a.aktiv
buchtausch_app-# ORDER BY entfernung_km, bu.titel LIMIT 5;

      titel       |  stadt  | entfernung_km
------------------+---------+---------------
 Der Winterpfad   | Berlin  |           0.5
 Die Uhrmacherin  | Leipzig |         148.5
 Der Winterpfad   | Dresden |         164.7
 Die Uhrmacherin  | Dresden |         164.7
 Pixel und Papier | Rostock |         195.4
(5 rows)
```

Konsistenzpruefung (erwartet: 0 Verstoesse)

psql - buchtausch_app - Konsistenzpruefung (erwartet: 0 Verstoesse)

```
buchtausch_app=# SELECT 'mehrere offene Ausleihen je Exemplar' AS regel, count(*) AS verstoesse FROM (
buchtausch_app-#   SELECT exemplar_id FROM ausleihvorgang WHERE status IN ('aktiv','ueberfaellig')
buchtausch_app-#   GROUP BY exemplar_id HAVING count(*) > 1) x
buchtausch_app-# UNION ALL SELECT 'Leihdauer ueber max_leihdauer_tage', count(*)
buchtausch_app-#   FROM ausleihvorgang av JOIN exemplar e USING (exemplar_id)
buchtausch_app-#   WHERE av.geplantes_rueckgabedatum - av.ausleihdatum > e.max_leihdauer_tage
buchtausch_app-# UNION ALL SELECT 'Zeitfenster gehoert nicht zum Besitzer', count(*)
buchtausch_app-#   FROM ausleihvorgang av JOIN exemplar e USING (exemplar_id)
buchtausch_app-#   JOIN verfuegbarkeitszeitfenster z USING (zeitfenster_id) WHERE z.benutzer_id <> e.besitzer_id
buchtausch_app-# UNION ALL SELECT 'Bewertung ohne zurueckgegebene Ausleihe', count(*)
buchtausch_app-#   FROM bewertung b JOIN ausleihvorgang av USING (ausleihvorgang_id)
buchtausch_app-#   WHERE av.status <> 'zurueckgegeben';

                  regel                  | verstoesse
-----------------------------------------+------------
 mehrere offene Ausleihen je Exemplar    |          0
 Leihdauer ueber max_leihdauer_tage      |          0
 Zeitfenster gehoert nicht zum Besitzer  |          0
 Bewertung ohne zurueckgegebene Ausleihe |          0
(4 rows)
```

6. Ergebnisse: Negativtests

Jeder Negativtest wird in einer Transaktion mit ROLLBACK ausgeführt und muss mit einer Fehlermeldung scheitern. Es werden keine Daten verändert (nach allen Tests: unverändert 342 Datensätze).

Selbst-Ausleihe · Regel: Eigenes Exemplar nicht ausleihbar; abgesichert durch: Trigger

psql - buchtausch_app - Selbst-Ausleihe

```
buchtausch_app=# INSERT INTO ausleihvorgang (exemplar_id, entleiher_id, zeitfenster_id, ausleihdatum, geplantes_rueckgabedatum)
buchtausch_app-# VALUES (1, 1, 1, '2026-10-28', '2026-11-04');

ERROR:  Ein Exemplar kann nicht von der besitzenden Person selbst ausgeliehen werden (exemplar_id=1).
CONTEXT:  PL/pgSQL function fn_check_ausleihvorgang() line 11 at RAISE
```

Bewertung vor Abschluss · Regel: Nur abgeschlossene Ausleihen bewertbar; abgesichert durch: Trigger

psql - buchtausch_app - Bewertung vor Abschluss

```
buchtausch_app=# INSERT INTO bewertung (ausleihvorgang_id, sternebewertung, kommentar)
buchtausch_app-# VALUES (28, 5, 'Ausleihe 28 ist noch aktiv');

ERROR:  Bewertung nur fuer abgeschlossene (zurueckgegebene) Ausleihvorgaenge moeglich (ausleihvorgang_id=28).
CONTEXT:  PL/pgSQL function fn_check_bewertung_nur_abgeschlossen() line 9 at RAISE
```

Doppelte Ausleihe · Regel: Ein Exemplar nicht mehrfach gleichzeitig offen; abgesichert durch: Partieller UNIQUE-Index

psql - buchtausch_app - Doppelte Ausleihe

```
buchtausch_app=# INSERT INTO ausleihvorgang (exemplar_id, entleiher_id, zeitfenster_id, ausleihdatum, geplantes_rueckgabedatum)
buchtausch_app-# VALUES (24, 1, 24, '2026-10-28', '2026-11-05');

ERROR:  duplicate key value violates unique constraint "ux_ausleihvorgang_offen_je_exemplar"
DETAIL:  Key (exemplar_id)=(24) already exists.
```

Max. Leihdauer ueberschritten · Regel: Geplante Dauer <= max_leihdauer_tage; abgesichert durch: Trigger

psql - buchtausch_app - Max. Leihdauer ueberschritten

```
buchtausch_app=# INSERT INTO ausleihvorgang (exemplar_id, entleiher_id, zeitfenster_id, ausleihdatum, geplantes_rueckgabedatum)
buchtausch_app-# VALUES (5, 1, 5, '2026-10-28', '2026-11-28');

ERROR:  Geplante Leihdauer (31 Tage) ueberschreitet die maximale Leihdauer des Exemplars (10 Tage).
CONTEXT:  PL/pgSQL function fn_check_ausleihvorgang() line 16 at RAISE
```

Ungueltige Koordinaten · Regel: Breitengrad -90..90, Laengengrad -180..180; abgesichert durch: CHECK-Constraint

psql - buchtausch_app - Ungueltige Koordinaten

```
buchtausch_app=# INSERT INTO abholort (benutzer_id, bezeichnung, strasse, plz, stadt, breitengrad, laengengrad)
buchtausch_app-# VALUES (1, 'Testort', 'Weg 1', '10115', 'Berlin', 95.0, 13.4);

ERROR:  new row for relation "abholort" violates check constraint "chk_abholort_breitengrad"
DETAIL:  Failing row contains (26, 1, Testort, Weg 1, 10115, Berlin, 95.000000, 13.400000).
```

Zeitfenster eines anderen Besitzers · Regel: Zeitfenster gehoert dem Besitzer des Exemplars; abgesichert durch: Trigger

psql - buchtausch_app - Zeitfenster eines anderen Besitzers

```
buchtausch_app=# INSERT INTO ausleihvorgang (exemplar_id, entleiher_id, zeitfenster_id, ausleihdatum, geplantes_rueckgabedatum)
buchtausch_app-# VALUES (6, 1, 2, '2026-10-28', '2026-11-04');

ERROR:  Zeitfenster 2 gehoert nicht zur besitzenden Person von Exemplar 6.
CONTEXT:  PL/pgSQL function fn_check_ausleihvorgang() line 22 at RAISE
```

Zeitfenster nicht als Angebot eingerichtet · Regel: Ausleihe nur ueber ein angebotenes Zeitfenster; abgesichert durch: Trigger

psql - buchtausch_app - Zeitfenster nicht als Angebot eingerichtet

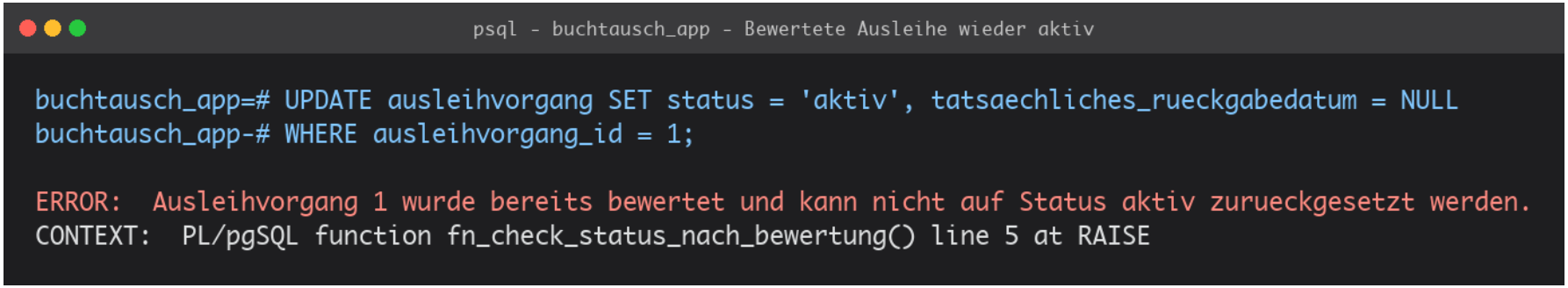
```
buchtausch_app=# INSERT INTO ausleihvorgang (exemplar_id, entleiher_id, zeitfenster_id, ausleihdatum, geplantes_rueckgabedatum)
buchtausch_app-# VALUES (7, 1, 31, '2026-10-28', '2026-11-04');

ERROR:  Zeitfenster 31 ist fuer Exemplar 7 nicht als Angebot eingerichtet.
CONTEXT:  PL/pgSQL function fn_check_ausleihvorgang() line 28 at RAISE
```

Angebot mit fremdem Abholort · Regel: Abholort/Zeitfenster gehoeren dem Besitzer; abgesichert durch: Trigger



Bewertete Ausleihe wieder aktiv · Regel: Bewertete Ausleihe bleibt abgeschlossen; abgesichert durch: Trigger



zurueckgegeben ohne Rueckgabedatum · Regel: status 'zurueckgegeben' <=> Rueckgabedatum vorhanden; abgesichert durch: CHECK-Constraint

psql - buchtausch_app - zurueckgegeben ohne Rueckgabedatum

```
buchtausch_app=# UPDATE ausleihvorgang SET status = 'zurueckgegeben'
buchtausch_app-# WHERE ausleihvorgang_id = 28;

ERROR:  new row for relation "ausleihvorgang" violates check constraint "chk_ausleih_status_rueckgabe"
DETAIL:  Failing row contains (28, 26, 4, 15, 2026-10-22, 2026-11-05, null, zurueckgegeben).
```

Sternebewertung 6 · Regel: Sterne 1 bis 5; abgesichert durch: CHECK-Constraint

psql - buchtausch_app - Sternebewertung 6

```
buchtausch_app=# INSERT INTO bewertung (ausleihvorgang_id, sternebewertung, erstellt_am)
buchtausch_app-# VALUES (21, 6, '2026-10-28');

ERROR:  new row for relation "bewertung" violates check constraint "bewertung_sternebewertung_check"
DETAIL:  Failing row contains (25, 21, 6, null, 2026-10-28).
```

Doppelte E-Mail-Adresse · Regel: E-Mail eindeutig; abgesichert durch: UNIQUE-Constraint

psql - buchtausch_app - Doppelte E-Mail-Adresse

```
buchtausch_app=# INSERT INTO benutzer (vorname, nachname, email)
buchtausch_app-# VALUES ('Test', 'Person', 'lena.vogt@example.com');

ERROR:  duplicate key value violates unique constraint "benutzer_email_key"
DETAIL:  Key (email)=(lena.vogt@example.com) already exists.
```

Zeitfenster Ende vor Beginn · Regel: endzeit > startzeit; abgesichert durch: CHECK-Constraint

psql - buchtausch_app - Zeitfenster Ende vor Beginn

```
buchtausch_app=# INSERT INTO verfuegbarkeitszeitfenster (benutzer_id, wochentag, startzeit, endzeit)
buchtausch_app-# VALUES (1, 'Montag', '12:00', '10:00');

ERROR:  new row for relation "verfuegbarkeitszeitfenster" violates check constraint "chk_zeitfenster_reihenfolge"
DETAIL:  Failing row contains (38, 1, Montag, 12:00:00, 10:00:00).
```

Erscheinungsjahr in der Zukunft · Regel: Erscheinungsjahr nicht in der Zukunft (zeitabhaengig); abgesichert durch: Trigger (statt CHECK)

psql - buchtausch_app - Erscheinungsjahr in der Zukunft

```
buchtausch_app=# INSERT INTO buch (titel, erscheinungsjahr, sprache_id)
buchtausch_app-# VALUES ('Zukunftsroman', 2090, 1);

ERROR:  erscheinungsjahr 2090 liegt zu weit in der Zukunft (maximal naechstes Jahr erlaubt).
CONTEXT:  PL/pgSQL function fn_check_erscheinungsjahr() line 5 at RAISE
```

Besitzerwechsel eines Exemplars · Regel: besitzer_id unveraenderlich; abgesichert durch: Trigger

psql - buchtausch_app - Besitzerwechsel eines Exemplars

```
buchtausch_app=# UPDATE exemplar SET besitzer_id = 2 WHERE exemplar_id = 1;

ERROR:  Spalte exemplar.besitzer_id ist unveraenderlich (Aenderung wuerde Zuordnungen von Angeboten/Ausleihen ungueltig machen).
CONTEXT:  PL/pgSQL function fn_spalte_unveraenderlich() line 4 at RAISE
```

Zweite Hauptadresse · Regel: Hoechstens eine Hauptadresse je Person; abgesichert durch: Partieller UNIQUE-Index

psql - buchtausch_app - Zweite Hauptadresse

```
buchtausch_app=# INSERT INTO adresse (benutzer_id, strasse, hausnummer, plz, stadt, ist_hauptadresse)
buchtausch_app-# VALUES (1, 'Testweg', '1', '10115', 'Berlin', TRUE);

ERROR:  duplicate key value violates unique constraint "ux_adresse_hauptadresse"
DETAIL:  Key (benutzer_id)=(1) already exists.
```

Leihdauer 0 Tage · Regel: max_leihdauer_tage zwischen 1 und 365; abgesichert durch: CHECK-Constraint

psql - buchtausch_app - Leihdauer 0 Tage

```
buchtausch_app=# INSERT INTO exemplar (buch_id, besitzer_id, zustand, max_leihdauer_tage)
buchtausch_app-# VALUES (1, 1, 'gut', 0);

ERROR:  new row for relation "exemplar" violates check constraint "exemplar_max_leihdauer_tage_check"
DETAIL:  Failing row contains (35, 1, 1, gut, f, 0, 2026-11-05).
```

Doppeltes Angebot · Regel: Gleiche Kombination Exemplar/Abholort/Zeitfenster nur einmal; abgesichert durch: UNIQUE-Constraint

psql - buchtausch_app - Doppeltes Angebot

```
buchtausch_app=# INSERT INTO angebot (exemplar_id, abholort_id, zeitfenster_id)
buchtausch_app-# VALUES (1, 1, 1);

ERROR:  duplicate key value violates unique constraint "angebot_exemplar_id_abholort_id_zeitfenster_id_key"
DETAIL:  Key (exemplar_id, abholort_id, zeitfenster_id)=(1, 1, 1) already exists.
```

Rueckgabedatum vor Ausleihdatum · Regel: geplantes Rueckgabedatum nach Ausleihdatum; abgesichert durch: CHECK-Constraint

psql - buchtausch_app - Rueckgabedatum vor Ausleihdatum

```
buchtausch_app=# INSERT INTO ausleihvorgang (exemplar_id, entleiher_id, zeitfenster_id, ausleihdatum, geplantes_rueckgabedatum)
buchtausch_app-# VALUES (9, 1, 9, '2026-10-28', '2026-10-20');

ERROR:  new row for relation "ausleihvorgang" violates check constraint "chk_ausleih_rueckgabe_reihenfolge"
DETAIL:  Failing row contains (42, 9, 1, 9, 2026-10-28, 2026-10-20, null, aktiv).
```

Zweite Bewertung derselben Ausleihe · Regel: Hoechstens eine Bewertung je Ausleihvorgang; abgesichert durch: UNIQUE-Constraint

psql - buchtausch_app - Zweite Bewertung derselben Ausleihe

```
buchtausch_app=# INSERT INTO bewertung (ausleihvorgang_id, sternebewertung)
buchtausch_app-# VALUES (1, 4);

ERROR:  duplicate key value violates unique constraint "bewertung_ausleihvorgang_id_key"
DETAIL:  Key (ausleihvorgang_id)=(1) already exists.
```

Nicht existierendes Buch · Regel: Referenzielle Integritaet; abgesichert durch: FOREIGN KEY

psql - buchtausch_app - Nicht existierendes Buch

```
buchtausch_app=# INSERT INTO exemplar (buch_id, besitzer_id, zustand, max_leihdauer_tage)
buchtausch_app-# VALUES (9999, 1, 'gut', 14);

ERROR:  insert or update on table "exemplar" violates foreign key constraint "exemplar_buch_id_fkey"
DETAIL:  Key (buch_id)=(9999) is not present in table "buch".
```